

DeclaraAI

Agentic RAG para apoio à declaração de IRPF



Instituição: Universidade do Estado do Amazonas
Disciplina: Oficina e Desenvolvimento de Sistemas I
Projeto: Micro SaaS com Agentic RAG
Data: 3 de junho de 2026
Integrantes: Juliana Ballin Lima
Fernando Luiz Da Silva Freire

Manaus, AM

Sumário

1	Resumo Executivo	1
2	Definição do Problema	1
3	Arquitetura da Solução	1
3.1	Visão Geral	1
3.2	Pipeline RAG	2
3.3	Comportamento Agentic	2
4	Base de Conhecimento	2
5	Modelo de Linguagem e Recuperação	2
6	Interface	3
7	Avaliação e Comparações	3
8	Etapas de Desenvolvimento	4
9	Limitações e Riscos	4
10	Conclusão	4

1 Resumo Executivo

O DeclaraAI é um assistente inteligente para apoiar contribuintes brasileiros na organização de documentos e no entendimento de regras da declaração do Imposto de Renda Pessoa Física. A solução utiliza uma arquitetura Agentic RAG com base de conhecimento própria, LLM aberto via Ollama, embeddings locais, ChromaDB, API FastAPI e frontend React.

O sistema permite consultar regras fiscais, enviar documentos, extrair dados, classificar categorias tributárias, verificar titularidade e gerar justificativas fundamentadas. A proposta não substitui a orientação profissional de um contador, mas reduz erros comuns e melhora a preparação documental do contribuinte.

2 Definição do Problema

A declaração do IRPF envolve regras, limites, documentos comprobatórios e categorias que mudam com o ano fiscal. Usuários leigos frequentemente enfrentam três dificuldades:

- identificar se uma despesa é dedutível;
- organizar documentos aceitos pela Receita Federal;
- entender quando uma informação está fora da base de conhecimento.

O público-alvo são contribuintes pessoas físicas com perfil simples ou intermediário, especialmente usuários que acumulam recibos, notas fiscais, informes de rendimentos e comprovantes educacionais ou médicos durante o ano.

3 Arquitetura da Solução

3.1 Visão Geral

Camada	Responsabilidade
Frontend React	Interface de chat, upload, base de conhecimento, histórico, avaliação e status.
FastAPI	Endpoints REST, validações, orquestração de serviços e documentação Swagger.
Pipeline RAG	Ingestão, chunking, embeddings, recuperação, re-ranking e geração.
Ollama	Execução local do LLM aberto usado em geração e classificação.
ChromaDB	Persistência vetorial dos chunks recuperáveis.
SQLite	Histórico de documentos salvos e dados estruturados.

3.2 Pipeline RAG

O pipeline segue oito etapas:

- 1. carregamento de documentos da base;
- 2. extração e limpeza textual;
- 3. divisão em chunks de 600 caracteres com 80 caracteres de sobreposição;
- 4. geração de embeddings multilíngues;
- 5. indexação no ChromaDB;
- 6. recuperação semântica dos trechos;
- 7. re-ranking com CrossEncoder;
- 8. geração de resposta com LLM local.

3.3 Comportamento Agentic

O agente decide qual ferramenta usar conforme a intenção do usuário. Perguntas abertas acionam busca vetorial. Uploads acionam extração, classificação, verificação de titularidade e justificativa enriquecida. Consultas sobre documentos salvos usam histórico e metadados.

4 Base de Conhecimento

Documento	Tipo	Justificativa
guia_imposto_renda.txt	TXT	Reúne regras resumidas de obrigatoriedade, deduções e documentos comuns do IRPF.
pr-irpf-2024.pdf	PDF	Documento oficial de perguntas e respostas da Receita Federal, usado como fonte primária.

Essas fontes cobrem despesas médicas, educação, previdência privada, rendimentos, dependentes, aluguéis, prazos e penalidades. A base pode ser ampliada pela interface.

5 Modelo de Linguagem e Recuperação

O modelo padrão é o `mistral` executado via Ollama. A escolha prioriza licença aberta, execução local, bom desempenho em português e custo computacional compatível com máquinas comuns. O embedding padrão é `sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2`, escolhido por ser leve, multilíngue e adequado para CPU.

Decisão	Motivo
Ollama local	Evita envio de documentos fiscais para APIs externas.
ChromaDB	Banco vetorial simples, persistente e suficiente para protótipo.
Re-ranking	Melhora a ordem dos trechos em perguntas complexas.
Temperatura baixa	Reduz alucinações em domínio fiscal.

6 Interface

O frontend foi desenvolvido em React + Vite. A interface possui páginas de uso direto:

- Chat RAG com fontes consultadas;
- Upload de documentos com revisão antes de salvar;
- Base de conhecimento para adicionar ou remover fontes;
- Histórico de documentos salvos;
- Avaliação do pipeline;
- Status do sistema, incluindo Ollama, modelos e chunks indexados.

7 Avaliação e Comparações

O dataset `data/eval/perguntas.json` possui 60 perguntas anotadas com resposta de referência, palavras-chave e dificuldade. As avaliações salvam resultados em CSV para comparação final.

Script	Objetivo
<code>scripts/avaliar_llm.py</code>	Comparar modelos Ollama e executar ablation study sem RAG.
<code>scripts/avaliar_chunking.py</code>	Comparar configurações de chunking e latência.
<code>scripts/avaliar_ragas.py</code>	Calcular métricas RAGAS com Ollama local como juiz.

Métricas implementadas:

- taxa de recuperação;
- score médio de contexto;
- cobertura de palavras-chave;
- latência por pergunta;
- casos sem contexto;
- métricas RAGAS opcionais.

8 Etapas de Desenvolvimento

Etapa	Status	Resultado
Base RAG	Concluída	Ingestão, chunking, ChromaDB, embeddings e resposta contextualizada.
Classificação	Concluída	LLM-first com fallback por regras e origem da classificação.
Justificativa	Concluída	Resposta personalizada com trechos da base.
Frontend	Concluída	Interface React profissional e responsiva.
Avaliação	Concluída	Dataset de 60 perguntas e scripts de comparação.
Relatório	Concluída	Documentação técnica em LaTeX.
Busca híbrida	Futuro	BM25 + vetorial + fusão de rankings.
Notebooks	Futuro	Gráficos para artigo e apresentação.

9 Limitações e Riscos

- O sistema depende do Ollama e dos modelos instalados localmente.
- Regras fiscais mudam anualmente e exigem atualização da base.
- OCR pode falhar em imagens de baixa qualidade.
- A classificação pode exigir revisão do usuário em documentos atípicos.
- O projeto é apoio informativo e não substitui contador.

10 Conclusão

O DeclaraAI atende aos requisitos centrais da atividade: domínio bem definido, base de conhecimento própria, ingestão, chunking, embeddings, recuperação, LLM aberto, Agentic RAG, interface web, avaliação e documentação. A principal evolução futura é ampliar a avaliação experimental com busca híbrida, comparação de embeddings e dataset anotado de documentos reais ou sintéticos.